

Bitacora de uso de IA

Proyecto Centinela . Fase 3 — Pipeline, aceleracion y despliegue en el borde

Redes Neuronales — Deep Learning . Maestria en Ciencia de Datos, Universidad Santo Tomas
Grupo: Juan Manuel Gomez . Camilo Andres Martinez . Sergio Beltran

La Fase 3 se construyo con apoyo de IA generativa. El grupo declara que **comprende, verifico y puede defender** cada decision de framework, pipeline, optimizacion y despliegue. Se registran las interacciones relevantes, priorizando los casos **modificados o rechazados**. El caso 9 responde directamente a la retroalimentacion de la Fase 2 (circularidad etiqueta-entradas). Dos de ellos son **auditorias medidas** reproducidas en el Anexo A del notebook.

#	Que le pedimos a la IA	Que propuso	Decision	Justificacion y verificacion
1	Aumentos de datos para la huella visual.	El catalogo clasico: <code>RandomHorizontalFlip</code> , <code>RandomRotation</code> , <code>RandomResizedCrop</code> .	RECHAZADA	En una reticula calendario , espejar las columnas invierte el tiempo (una caida al final se vuelve caida al inicio: cambia la etiqueta real) y rotar/recortar destruye la posicion ano×mes, que ES la semantica. Se sustituyo por 4 aumentos que la respetan — jitter suave, desenfoque, desfase $\leq \frac{1}{2}$ mes y borrado aleatorio $\approx$ lecturas faltantes — mas Mixup como tecnica avanzada.
2	Exportar el modelo a ONNX.	<code>torch.onnx.export(...)</code> con los valores por defecto.	RECHAZADA	En PyTorch ≥ 2.13 el exportador por defecto (dynamo) exige la dependencia extra <code>onnxscript</code> y produce un grafo que el cuantizador de ONNX Runtime rechaza (<code>ShapeInferenceError</code>). Correccion: la ruta estable <code>dynamo=False</code> con <code>input_names/</code> <code>output_names/</code> <code>dynamic_axes</code> , con la que exportacion, paridad (<code>np.allclose</code> , <code>atol=1e-4</code>) y cuantizacion funcionan de punta a punta. Auditoria medida (Anexo A, caso 1).
3	Comprimir el modelo para el borde.	<code>torch.quantization.quantize_dynamic(modelo, {nn.Linear})</code> .	RECHAZADA	Medido: el archivo paso de 113 KB a solo 102 KB (~10 %) — esa API solo cuantiza capas <code>Linear</code> ; las convoluciones, donde

				vive el grueso de los pesos de una CNN, quedan en FP32. Correccion: INT8 sobre el grafo ONNX completo: 111 KB → 35 KB (–68 %) con –0,2 pt de exactitud. Auditoria medida (Anexo A, caso 2).
4	Configurar el DataLoader.	Receta estandar: <code>num_workers=4</code> , <code>prefetch_factor</code> , <code>pin_memory</code> .	MODIFICADA	Medido en el pipeline real: con imagenes de 64 px el overhead de los procesos trabajadores supero su beneficio en CPU (0,70 vs 0,57 s/epoca con workers=0). Se dejo configurable y documentado: el paralelismo paga cuando el costo por lote lo justifica (en Colab-GPU, workers + pin_memory si solapan CPU/GPU).
5	Comparar tf.data y DataLoader en el mismo cuaderno.	Cargar ambos frameworks en el mismo runtime.	MODIFICADA	Riesgo de colision de runtimes y doble consumo de memoria — la advertencia explicita de la ficha. El benchmark de <code>tf.data</code> (y la paridad de la implementacion Keras) se aislaron en subprocesos con reporte por JSON/NPZ: el mismo patron de aislamiento que se usaria en produccion.
6	Activar precision mixta.	Asumir que FP16 acelera en cualquier hardware.	MODIFICADA	Medido: en CPU, bfloat16 no acelera (×0,81, sin Tensor Cores) y el RSS apenas baja porque AMP conserva los pesos maestros en FP32 (el matiz de la ficha). El bucle quedo <i>device-aware</i> (FP16+GradScaler en GPU / bfloat16 en CPU) y reporta automaticamente el caso GPU al re-ejecutarse en Colab-T4.
7	Estrategia de resiliencia del entrenamiento.	Checkpointing cada 5 epocas.	ACEPTADA	Implementado y extendido con reanudacion: si el cuaderno encuentra el checkpoint final de una variante, reanuda desde el (demostrado en la ejecucion adjunta) — la politica exacta

				para las desconexiones de Colab y la gestion de cuota.
8	Citar el tamaño de los backbones de comparacion.	Escribir "25,6 M / 3,5 M parametros" como cifras de referencia habituales.	RECHAZADA	Son las cifras <i>de catalogo</i> que no coinciden con la arquitectura instanciada: el conteo REAL es ResNet50 23,6 M y MobileNetV2 2,26 M (sin capa top, 96 px). Atendiendo la retroalimentacion de la Fase 2 (texto que no coincide con lo que imprime el codigo), se ELIMINARON todas las cifras escritas a mano: el cuaderno calcula e imprime cada conteo con <code>count_params()</code> / <code>numel()</code> y el texto los toma de ahi.
9	Tratar la circularidad etiqueta-entradas señalada en la Fase 2.	Mantener la afirmacion de que el etiquetado no tiene circularidad.	RECHAZADA	La retroalimentacion de la Fase 2 demostro que la etiqueta se calcula con una regla sobre el mismo consumo que ven las redes, asi que un AUC alto recupera una regla conocida mas que detectar. En vez de negarlo, la Fase 3 lo declara y lo mide: cuantifica la zona gris (~1.860 usuarios) y la fragilidad de la regla (~184 etiquetas cambian con +/-10% de ruido), define el aporte del modelo como una frontera suave en esa zona, y condiciona todo uso operativo a la validacion contra actas reales. Es la ruta de mayor impacto que marco el docente.
10	Reportar el efecto de la precision mixta (FP16) en GPU.	Asumir que FP16 acelera el entrenamiento, como dice la teoria general.	MODIFICADA	Medido en la GPU real (RTX 4050): la VRAM baja 26% (75->55 MB, como predice la teoria), pero el tiempo NO mejora (2,86s FP32 vs 3,41s FP16, x0,84). En vez de afirmar una aceleracion inexistente, lo explicamos: con 27.939 parametros y 64 px, el overhead de conversion FP16/FP32 pesa mas que los Tensor Cores; la aceleracion temporal aplica a modelos grandes, no a redes <i>edge</i>. Es el mismo

escepticismo hacia lo que
confirma la teoría que pidió
la retroalimentación de la
Fase 2.

Declaración de integridad. Los integrantes están en capacidad de explicar sin notas: por qué se rechazaron los aumentos clásicos en la huella, por qué la ruta estable de exportación, por qué la cuantización dinámica de PyTorch no comprime una CNN, cuando pagan los workers del DataLoader, por qué AMP no acelera en CPU, y las razones de ingeniería de la partición borde/nube. La IA se usó como herramienta de construcción y auditoría, no como sustituto del criterio propio.